Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

Redes de Computadores I

Laboratório 4

Wireshark – DNS Query em Python

Realizado por: Diogo Carvalho nº 90247

PL: 2

Docente: Rui Filipe Rosa Paulo

Ano Letivo: 2025/2026

Relatório

Codigo

DNS query.py

```
import socket

import struct

import random


dnsserver = "10.10.22.228"

portnumber = 53


# transforma o domínio no formato usado pelo DNS

# exemplo: ualg.pt -> 04 ualg 02 pt 00

def encode_name(domain):

    parts = domain.split(".")

    name = b""


    for part in parts:

        name += struct.pack("!B", len(part))

        name += part.encode()


    name += b"\x00"

    return name


# cria a mensagem da query

def build_query(domain):

    transaction_id = random.randint(0, 65535)
```

```
# standard query + recursion desired
flags = 0x0100

# 1 pergunta, resto a 0
qdcount = 1
ancount = 0
nscount = 0
arcount = 0

# header DNS tem sempre 12 bytes
header = struct.pack(
    "!HHHHHH",
    transaction_id,
    flags,
    qdcount,
    ancount,
    nscount,
    arcount
)

# question section
qname = encode_name(domain)
qtype = 1    # A
qclass = 1   # IN

question = qname + struct.pack("!HH", qtype, qclass)
```

```
    return transaction_id, header + question


# lê nomes dentro da mensagem DNS
def read_name(message, offset):
    labels = []

    while True:
        length = message[offset]

        # se começar por 11, é um pointer
        if (length & 0xC0) == 0xC0:
            pointer = struct.unpack("!H", message[offset:offset +
2])[0]
            pointer = pointer & 0x3FFF
            name, _ = read_name(message, pointer)
            labels.append(name)
            offset += 2
            break

        # 0 quer dizer fim do nome
        if length == 0:
            offset += 1
            break

        offset += 1
        labels.append(message[offset:offset + length].decode())
        offset += length

    return ".".join(labels), offset
```

```
# lê um resource record
def read_record(message, offset):
    name, offset = read_name(message, offset)

    rtype, rclass, ttl, rdlength = struct.unpack("!HHIH",
message[offset:offset + 10])
    offset += 10

    rdata_start = offset
    rdata = message[offset:offset + rdlength]
    offset += rdlength

    return {
        "name": name,
        "type": rtype,
        "class": rclass,
        "ttl": ttl,
        "rdlength": rdlength,
        "rdata": rdata,
        "rdata_start": rdata_start
    }, offset

# imprime um resource record
def print_record(record, message, number):
    print("\nRegisto", number)
    print("Nome:", record["name"])
    print("Tipo:", record["type"])
    print("Classe:", record["class"])
```

```
print("TTL:", record["ttl"])


# A -> endereço IPv4
if record["type"] == 1 and record["rdlength"] == 4:
    ip = socket.inet_ntoa(record["rdata"])
    print("IP:", ip)


# NS -> nome do servidor
elif record["type"] == 2:
    ns_name, _ = read_name(message, record["rdata_start"])
    print("NS:", ns_name)


# CNAME -> nome canónico
elif record["type"] == 5:
    cname, _ = read_name(message, record["rdata_start"])
    print("CNAME:", cname)


# MX -> preferência + servidor de mail
elif record["type"] == 15:
    preference = struct.unpack("!H", record["rdata"][:2])[0]
    mx_name, _ = read_name(message, record["rdata_start"] + 2)
    print("Preference:", preference)
    print("MX:", mx_name)


# outros tipos ficam em hexadecimal
else:
    print("RDATA:", record["rdata"].hex())
```

```
def main():
    domain = input("Domínio: ")

    query_id, query = build_query(domain)

    # socket UDP
    client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    client.settimeout(5)

    # envia a query
    client.sendto(query, (dnsserver, portnumber))

    # recebe a resposta
    response, address = client.recvfrom(4096)

    print("\nResposta de:", address)

    # header
    header = struct.unpack("!HHHHHH", response[:12])

    response_id = header[0]
    flags = header[1]
    qdcount = header[2]
    ancount = header[3]
    nscount = header[4]
    arcount = header[5]

    print("\n--- HEADER ---")
```

```
    print("ID:", hex(response_id))
    print("Flags:", hex(flags))
    print("Questions:", qdcount)
    print("Answer RRs:", ancount)
    print("Authority RRs:", nscount)
    print("Additional RRs:", arcount)

    offset = 12

    # question section
    print("\n--- QUESTION SECTION ---")
    for _ in range(qdcount):
        qname, offset = read_name(response, offset)
        qtype, qclass = struct.unpack("!HH", response[offset:offset +
4])
        offset += 4

        print("Nome:", qname)
        print("Tipo:", qtype)
        print("Classe:", qclass)

    # answer section
    print("\n--- ANSWER SECTION ---")
    for i in range(ancount):
        record, offset = read_record(response, offset)
        print_record(record, response, i + 1)

    # authority section
    print("\n--- AUTHORITY SECTION ---")
```

```
    for i in range(nscount):
        record, offset = read_record(response, offset)
        print_record(record, response, i + 1)

    # additional section
    print("\n--- ADDITIONAL SECTION ---")
    for i in range(arcount):
        record, offset = read_record(response, offset)
        print_record(record, response, i + 1)

    client.close()

if __name__ == "__main__":
    main()
```

Resultados

Execuçao do programa

```
carva@Carvalho:~/projetos$ python3 DNS_query.py
Domínio: ualg.pt

Resposta de: ('10.10.22.228', 53)

--- HEADER ---
ID: 0x5663
Flags: 0x8180
Questions: 1
Answer RRs: 5
Authority RRs: 13
Additional RRs: 9

--- QUESTION SECTION ---
Nome: ualg.pt
Tipo: 1
Classe: 1

--- ANSWER SECTION ---

Registo 1
Nome: ualg.pt
Tipo: 1
Classe: 1
TTL: 186
IP: 193.136.224.101

Registo 2
Nome: ualg.pt
Tipo: 1
Classe: 1
TTL: 186
IP: 193.136.228.10

Registo 3
Nome: ualg.pt
Tipo: 1
Classe: 1
TTL: 186
IP: 193.136.224.100

Registo 4
Nome: ualg.pt
Tipo: 1
Classe: 1
TTL: 186
IP: 193.136.228.11

Registo 5
Nome: ualg.pt
Tipo: 1
Classe: 1
TTL: 186
IP: 10.3.0.16
```

```
--- AUTHORITY SECTION ---

Registo 1
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: k.root-servers.net

Registo 2
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: i.root-servers.net

Registo 3
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: m.root-servers.net

Registo 4
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: e.root-servers.net

Registo 5
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: g.root-servers.net

Registo 6
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: c.root-servers.net

Registo 7
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: a.root-servers.net

Registo 8
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: l.root-servers.net

Registo 9
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: j.root-servers.net
```

```
Registo 10
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: f.root-servers.net

Registo 11
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: h.root-servers.net

Registo 12
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: d.root-servers.net

Registo 13
Nome:
Tipo: 2
Classe: 1
TTL: 316827
NS: b.root-servers.net
```

```
--- ADDITIONAL SECTION ---

Registo 1
Nome: k.root-servers.net
Tipo: 1
Classe: 1
TTL: 497803
IP: 193.0.14.129

Registo 2
Nome: k.root-servers.net
Tipo: 28
Classe: 1
TTL: 497803
RDATA: 200107fd0000000000000000000001

Registo 3
Nome: i.root-servers.net
Tipo: 28
Classe: 1
TTL: 497803
RDATA: 200107fe0000000000000000000053

Registo 4
Nome: m.root-servers.net
Tipo: 1
Classe: 1
TTL: 497803
IP: 202.12.27.33

Registo 5
Nome: m.root-servers.net
Tipo: 28
Classe: 1
TTL: 497803
RDATA: 20010dc30000000000000000000035

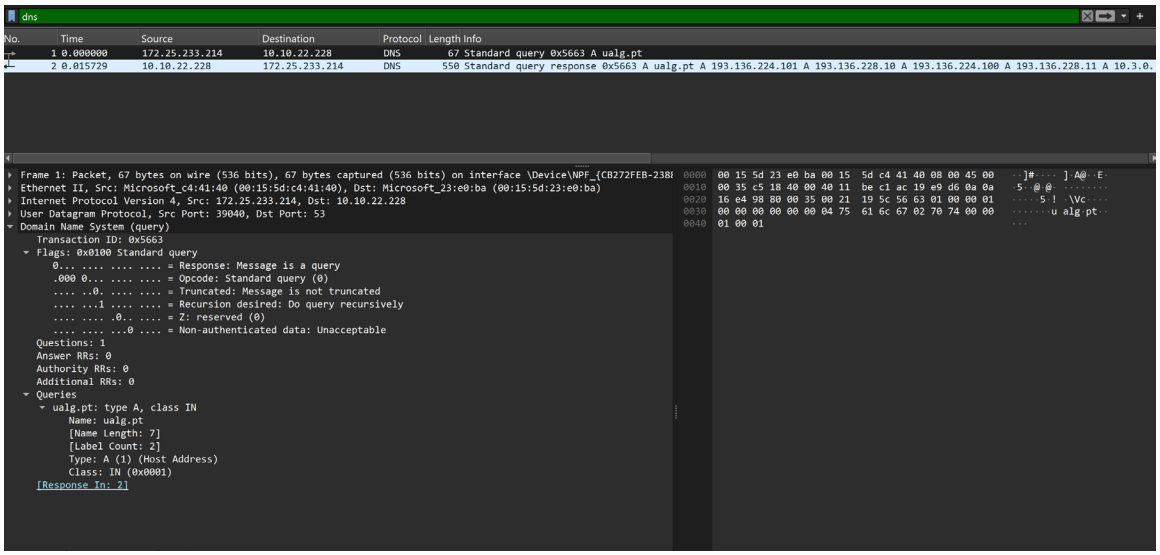
Registo 6
Nome: e.root-servers.net
Tipo: 1
Classe: 1
TTL: 497803
IP: 192.203.230.10

Registo 7
Nome: e.root-servers.net
Tipo: 28
Classe: 1
TTL: 497803
RDATA: 2001050000a800000000000000000e

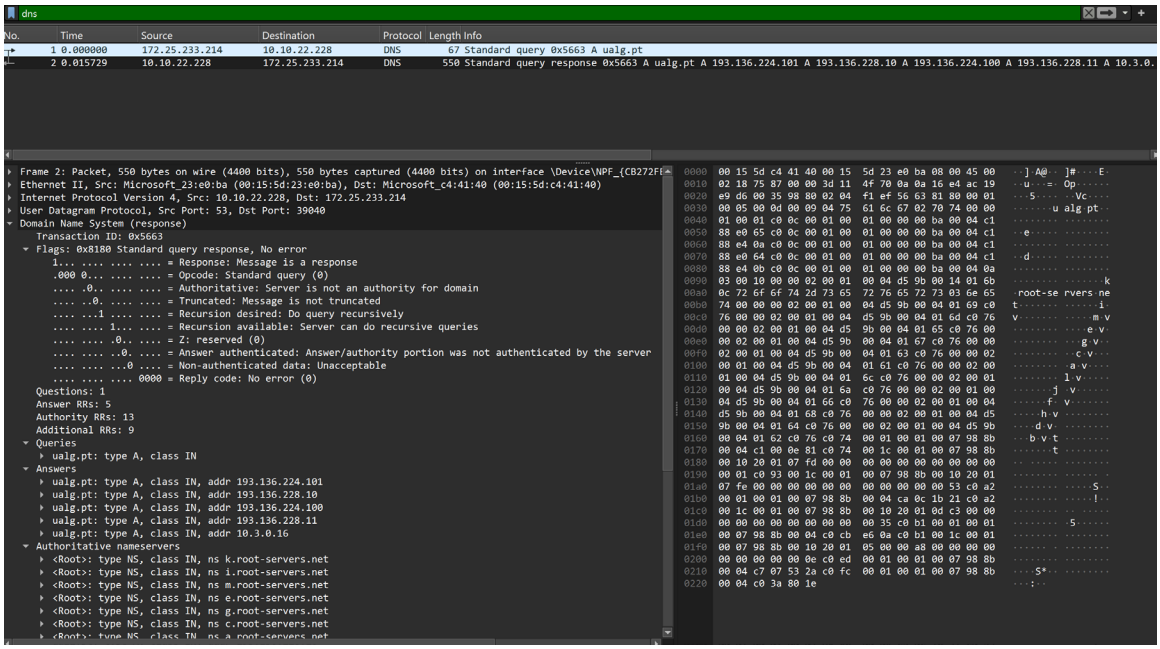
Registo 8
Nome: l.root-servers.net
Tipo: 1
Classe: 1
TTL: 497803
IP: 199.7.83.42

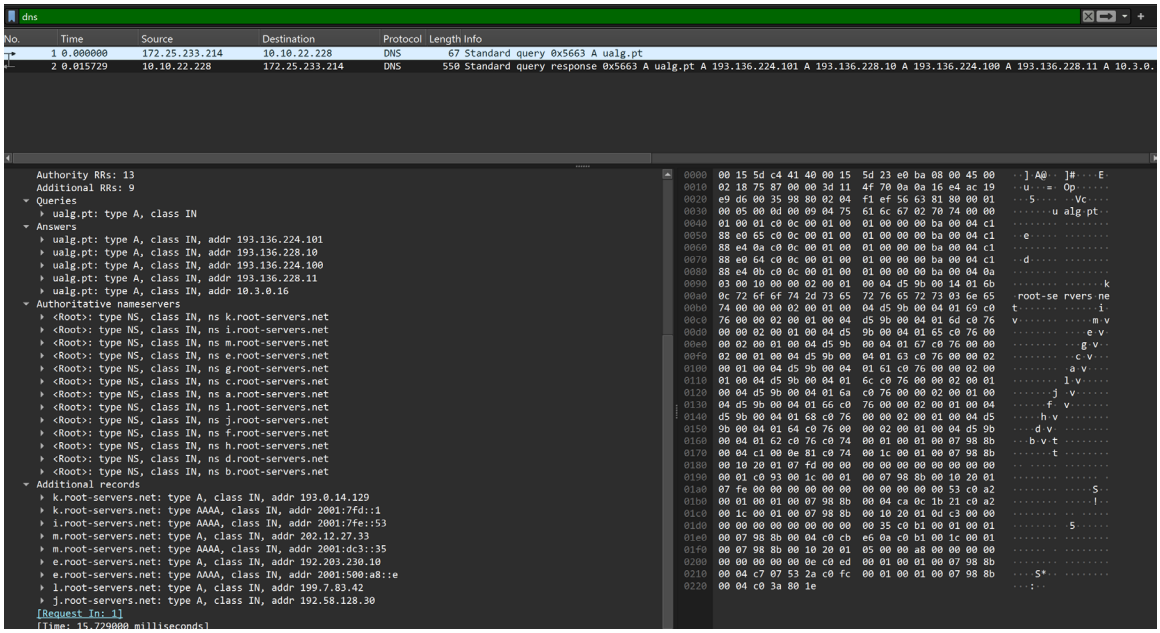
Registo 9
Nome: j.root-servers.net
Tipo: 1
Classe: 1
TTL: 497803
IP: 192.58.128.30
```

Mensagem enviada



Resposta




dns
No. Time Source Destination Protocol Length Info
1 0.000000 172.25.233.214 10.10.22.228 DNS 67 Standard query 0x5663 A ualg.pt
2 0.015729 10.10.22.228 172.25.233.214 DNS 550 Standard query response 0x5663 A ualg.pt A 193.136.224.101 A 193.136.228.10 A 193.136.224.100 A 193.136.228.11 A 10.3.0.
Authority RRs: 13
Additional RRs: 9
Queries
ualg.pt: type A, class IN
Answers
ualg.pt: type A, class IN, addr 193.136.224.101
ualg.pt: type A, class IN, addr 193.136.228.10
ualg.pt: type A, class IN, addr 193.136.224.100
ualg.pt: type A, class IN, addr 193.136.228.11
ualg.pt: type A, class IN, addr 10.3.0.16
Authoritative nameservers
<Root>: type NS, class IN, ns k.root-servers.net
<Root>: type NS, class IN, ns i.root-servers.net
<Root>: type NS, class IN, ns m.root-servers.net
<Root>: type NS, class IN, ns e.root-servers.net
<Root>: type NS, class IN, ns g.root-servers.net
<Root>: type NS, class IN, ns c.root-servers.net
<Root>: type NS, class IN, ns a.root-servers.net
<Root>: type NS, class IN, ns l.root-servers.net
<Root>: type NS, class IN, ns j.root-servers.net
<Root>: type NS, class IN, ns f.root-servers.net
<Root>: type NS, class IN, ns h.root-servers.net
<Root>: type NS, class IN, ns d.root-servers.net
<Root>: type NS, class IN, ns b.root-servers.net
Additional records
k.root-servers.net: type A, class IN, addr 193.0.14.129
k.root-servers.net: type AAAA, class IN, addr 2001:7fd::1
i.root-servers.net: type AAAA, class IN, addr 2001:7fe::53
m.root-servers.net: type A, class IN, addr 202.12.27.33
m.root-servers.net: type AAAA, class IN, addr 2001:dc3::35
e.root-servers.net: type A, class IN, addr 192.203.230.10
e.root-servers.net: type AAAA, class IN, addr 2001:500:a8::e
l.root-servers.net: type A, class IN, addr 199.7.83.42
j.root-servers.net: type A, class IN, addr 192.58.128.30
[Request In: 1]
[Time: 15.729000 milliseconds]
0000 00 15 5d c4 41 40 00 15 5d 23 e0 ba 08 00 45 00
0010 02 18 75 87 00 00 3d 11 4f 70 0a 0a 16 e4 ac 19
0020 e9 d6 00 35 98 80 02 04 f1 ef 56 63 81 80 00 01
0030 00 05 00 0d 00 09 04 75 61 6c 67 02 70 74 00 00
0040 01 00 01 c0 0c 00 01 00 01 00 00 00 ba 00 04 c1
0050 88 e0 65 c0 0c 00 01 00 01 00 00 00 ba 00 04 c1
0060 88 e4 0a c0 0c 00 01 00 01 00 00 00 ba 00 04 c1
0070 88 e0 64 c0 0c 00 01 00 01 00 00 00 ba 00 04 c1
0080 88 e4 0b c0 0c 00 01 00 01 00 00 00 ba 00 04 0a
0090 03 00 10 00 00 02 00 01 00 04 d5 9b 00 14 01 6b
00a0 0c 72 6f 6f 74 2d 73 65 72 76 65 72 73 03 6e 65
00b0 74 00 00 00 02 00 01 00 04 d5 9b 00 04 01 69 c0
00c0 76 00 00 02 00 01 00 04 d5 9b 00 04 01 6d c0 76
00d0 00 00 02 00 01 00 04 d5 9b 00 04 01 65 c0 76 00
00e0 00 02 00 01 00 04 d5 9b 00 04 01 67 c0 76 00 00
00f0 02 00 01 00 04 d5 9b 00 04 01 63 c0 76 00 00 02
0100 00 01 00 04 d5 9b 00 04 01 61 c0 76 00 00 02 00
0110 01 00 04 d5 9b 00 04 01 6c c0 76 00 00 02 00 01
0120 00 04 d5 9b 00 04 01 6a c0 76 00 00 02 00 01 00
0130 04 d5 9b 00 04 01 66 c0 76 00 00 02 00 01 00 04
0140 d5 9b 00 04 01 68 c0 76 00 00 02 00 01 00 04 d5
0150 9b 00 04 01 64 c0 76 00 00 02 00 01 00 04 d5 9b
0160 00 04 01 62 c0 76 c0 74 00 01 00 01 00 07 98 8b
0170 00 04 c1 00 0e 81 c0 74 00 1c 00 01 00 07 98 8b
0180 00 10 20 01 07 fd 00 00 00 00 00 00 00 00 00 01
0190 00 01 c0 93 00 1c 00 01 00 07 98 8b 00 10 20 01
01a0 07 fe 00 00 00 00 00 00 00 00 00 00 53 c0 a2
01b0 00 01 00 01 00 07 98 8b 00 04 ca 0c 1b 21 c0 a2
01c0 00 1c 00 01 00 07 98 8b 00 10 20 01 0d c3 00 00
01d0 00 00 00 00 00 00 00 00 00 35 c0 b1 00 01 00 01
01e0 00 07 98 8b 00 04 c0 cb e6 0a c0 b1 00 1c 00 01
01f0 00 07 98 8b 00 10 20 01 05 00 00 a8 00 00 00 00
0200 00 00 00 00 00 0e c0 ed 00 01 00 01 00 07 98 8b
0210 00 04 c7 07 53 2a c0 fc 00 01 00 01 00 07 98 8b
0220 00 04 c0 3a 80 1e